Functional & Performance Testing Template

Model Performance Test

Date	01.04.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

1. Functional Testing: UI Policy Enforcement

Objective: Verify that UI Policies correctly make fields mandatory based on specific conditions and prevent form submission when empty.

Test Case ID	Scenario	Expected Result	Status
FT-01	Trigger condition met (e.g., State = "In Progress").	Target fields immediately become mandatory (indicated by asterisk).	
FT-02	Trigger condition not met (e.g., State = "New").	Target fields remain optional or hidden as per policy.	
FT-03	Attempt to save record with empty mandatory fields.	Form submission is blocked; error message appears highlighting missing fields.	
FT-04	UI Policy Conflict (Multiple policies acting on one field).	The UI Policy with the highest **Order** number takes precedence.	
FT-05	Read-only vs. Mandatory.	If a field is Read-only and Mandatory, the user should be unable to submit if it's empty (check for logic loops).	

2. Performance Testing: UI Responsiveness

Objective: Ensure that UI Policies do not cause "browser lag" or delay form load times, especially on tables with high volumes of scripts.

Key Metrics to Monitor:

- **Form Load Time:** Time taken from clicking a record to the form being interactive.
- **Policy Execution Time:** Delay between changing a triggering field and the UI updating.

Test Case ID	Scenario	Expected Result
PT-01	**Client-Side Latency:** Rapidly toggle the triggering field 10 times.	UI responds in < 200ms without "hanging" or memory leaks.
PT-02	**Complex Conditions:** Use a UI Policy with 5+ conditions.	Evaluation of conditions does not noticeably delay form interaction.
PT-03	**Scripted Policies:** If using "Execute if true" scripts.	Scripts must be optimized to avoid synchronous server calls (e.g., avoid `glideRecord` in client scripts).

3. Migration Testing: Update Sets

Objective: Validate that the UI Policy and its associated actions/scripts migrate from **Development** to **Test/Prod** without corruption.

Pre-Migration Checklist

- [ ] Ensure all UI Policy Actions are captured in the **same** Update Set.
- [ ] Verify the Update Set is "Completed" before export.

Migration Scenarios

- **Preview Validation:** Check for "Local Changes" or "Collisions" during the Preview phase in the target instance.
- **Commit Verification:** Once committed, navigate to the UI Policy list.
  - *Check:* Is the `sys_id` consistent?
  - *Check:* Are the "UI Policy Actions" (the specific field behaviors) attached?
- **Functional Smoke Test:** Run **FT-01** and **FT-03** in the target environment immediately after migration.

4. Troubleshooting & Common Pitfalls

Note: If a UI Policy isn't working after migration, check the **Order** field. If another policy was created directly in the target environment with a higher order number, it may be overriding your migrated changes.

- **Global vs. View-Specific:** Ensure the UI Policy is set to the correct "View." If it’s set to "Mobile" but you are testing on "Desktop," it will not fire.
- **OnLoad vs. OnChange:** Verify if "OnLoad" is checked. If unchecked, the field will only become mandatory *after* a user changes a value, not when the form first opens.